

CS 699 · Reinforcement Learning · Spring 2026

Safe Automated Driving with Constrained Reinforcement Learning

Final Report

Reproducing & extending Kamran et al., IEEE ITSC 2022

Group 7

Bonian Han · Junyu Lu

github.com/BonianHan/CS699-SafeRL-Project

1. References and Acknowledgements

1.1 Reference of the paper

Kamran D., Simão T. D., Yang Q., Ponnambalam C. T., Fischer J., Spaan M. T. J., Lauer M. “A Modern Perspective on Safe Automated Driving for Different Traffic Dynamics Using Constrained Reinforcement Learning.” IEEE International Conference on Intelligent Transportation Systems (ITSC), 2022, pp. 4017–4023. DOI: 10.1109/ITSC55140.2022.9921907.

1.2 Links to resources

- Original paper code (TensorFlow 1.x): github.com/qisong-yang/SafeRLinHighwayEnvs
- Simulation environment: github.com/Farama-Foundation/HighwayEnv (highway-env 1.10.2, MIT licence)
- Safe-RL library: github.com/liuzuxin/FSRL (FSRL — discrete PPOLagrangian and CPO)
- RL framework: github.com/thu-ml/tianshou (Tianshou ~0.5.1)
- OmniSafe (audited but not used as a baseline; see §4.4): github.com/PKU-Alignment/omnisafe
- Our code, results, slides, and report: github.com/BonianHan/CS699-SafeRL-Project (branch milestone3)

1.3 Other references

- Schulman J. et al. “Proximal Policy Optimization Algorithms.” arXiv:1707.06347, 2017.
- Achiam J. et al. “Constrained Policy Optimization.” ICML 2017.
- Altman E. “Constrained Markov Decision Processes.” Chapman & Hall, 1999.
- Stooke A., Achiam J., Abbeel P. “Responsive Safety in RL by PID Lagrangian Methods.” ICML 2020.
- Yang T.-Y. et al. “WCSAC: Worst-Case Soft Actor-Critic for Safety-Constrained RL.” AAAI 2021.

1.4 Workload distribution

We worked as a 2-person team and split the work roughly evenly. Both members participated in scoping, debugging, and the final report; each owned a primary execution lane as listed below.

Member	Primary responsibilities	Secondary	% workload
Bonian Han	Codebase architecture (saferl_suite, train.py, run_roadmap), CostWrapper bug fix, decoupled-reward design, multi-scenario sweep orchestration, slide deck and report	Demo videos, statistics	50 %
Junyu Lu	Discrete-action FSRL adapter, CPO integration,	Slide	50 %

Member	Primary responsibilities	Secondary	% workload
	hyperparameter tuning for the merge replication, training-curve analysis, OmniSafe compatibility audit	drafting, GPU runs	

Table 1. Workload distribution between the two team members.

2. Application, MDP, and RL Model

2.1 Application introduction

The paper studies safe decision-making for an autonomous ego-vehicle that must interact with surrounding traffic in highway-env, an open-source Gymnasium simulator built on top of an IDM+MOBIL longitudinal-and-lane-change traffic model. Per simulation step ($\Delta t = 0.1$ s in the slow loop, 0.025 s for the underlying kinematic integrator), the agent receives a Kinematics observation and emits one discrete meta-action. Episodes terminate on collision, on reaching the goal, or after a fixed number of steps.

We focus on four scenarios from highway-env, all using the DiscreteMetaAction interface: merge-v0 (the paper's primary benchmark), highway-v0, roundabout-v0, and intersection-v1. Each provides a different traffic-dynamics regime, which is exactly the regime-shift problem the paper claims constrained RL solves more robustly than reward shaping.

2.2 MDP / CMDP formulation

We model the task as a Constrained Markov Decision Process (S, A, P, r, c, γ, d):

Component	Definition
State $s_t \in S$	Flattened Kinematics observation of the 5 nearest vehicles (8 for intersection-v1) — for each vehicle: [presence, x, y, vx, vy]. Dimension: 25 (40 for intersection-v1).
Action $a_t \in A$	5 discrete meta-actions {LANE_LEFT, IDLE, LANE_RIGHT, FASTER, SLOWER}. The low-level controller handles steering and throttle.
Transition P	Deterministic given the current world state — surrounding vehicles follow IDM (longitudinal) and MOBIL (lane-change) policies; ego dynamics follow a kinematic bicycle model. No explicit transition matrix; the simulator is treated as a black-box environment.
Reward r_t	high_speed_reward (0.4 weight) plus a lane-position bonus. Crucially, we set collision_reward = 0 so safety is NOT penalized through reward (see §4.4 ablation).
Cost c_t	1 if the ego collides at step t , else 0. This is the only safety signal. We added a Gym wrapper (CostWrapper) that maps highway-env's info["crashed"] to info["cost"], because the paper's open-source code reads info["cost"] which highway-env never sets.
Discount γ	0.99 (paper default). GAE- λ for advantage estimation: 0.97.
Constraint d	Cost limit $d = 0.1 \Rightarrow$ the constrained policy is required to keep the expected episodic collision rate below 10 %.

Table 2. CMDP components used throughout the project.

CMDP objective: $\max \mathbb{E}[\sum \gamma^t r_t]$ **subject to** $\mathbb{E}[\sum \gamma^t c_t] \leq d$.

Intuition for the value functions

Two critics are trained: $V(s)$ for return and $V_c(s)$ for cumulative cost. $V(s)$ answers “how much speed/lane reward will I collect from here on?”; $V_c(s)$ answers “how likely is a crash from here on, weighted by discount?”. For example, in merge-v0, a state where the ego is close behind a slow lead vehicle on the on-ramp has $V_c(s) \approx 0.6-0.8$ (high crash probability if the agent accelerates) and $V(s)$ only ≈ 7 (low expected reward). After PPOLag converges, the policy learns that braking from such a state lowers V_c at the cost of a small dip in V — exactly the trade-off the constraint encodes.

2.3 RL approach

We use three on-policy algorithms, all sharing the same MLP architecture (2 hidden layers $\times$ 64 units, tanh activation, orthogonal init, separate actor / reward-critic / cost-critic networks).

Algorithm	What it adds to PPO	Why include it
PPO	Vanilla PPO with clip $\epsilon = 0.2$, target KL = 0.01. Optimises only the reward objective; ignores the cost constraint.	Unconstrained baseline. If a constrained method does not beat PPO on safety, the framework adds nothing.
PPO-Lagrangian	Wraps PPO in a Lagrangian ($\max_{\pi} \min_{\lambda \geq 0} J_R(\pi) - \lambda(J_C(\pi) - d)$). λ is updated online by a PID controller ($K_p = 10$, $K_i = 1$, $K_d = 1$) from the running estimate of $J_C - d$.	The paper's headline method. λ adapts automatically: if cost is too high, λ rises and the policy gradient pushes toward safer actions; if cost is below d , λ shrinks and the policy can chase reward.
CPO	Trust-region update solving a constrained quadratic programme via conjugate gradient on the cost-advantage estimator. Provides per-update feasibility guarantees (in the limit of accurate estimates).	Different family of constrained method; tests whether the CMDP advantage holds across optimisation strategies, not just Lagrangian.

Table 3. Three algorithms compared, all sharing the same network architecture.

Compared to classical RL covered in class (DQN, vanilla policy gradient, actor-critic), the key extension is the explicit cost constraint and the Lagrangian dual variable. Conceptually, λ plays the role of a learned multiplicative penalty weight; the PID controller is what makes its update smooth instead of oscillatory. This lets us specify safety as an interpretable scalar (“crash rate $\leq 10\%$ ”) rather than as a brittle scalar weight that has to be re-tuned for every new traffic regime.



Figure 1. PPO and PPOLag training curves on merge-v0. PPOLag's cost crosses below $d = 0.1$ by epoch 5; PPO's cost oscillates between 0.2 and 0.7 with no incentive to descend.

3. Codebase, System, and Experiment Setup

3.1 Libraries and system setup

Library	Version	Role
Python	3.10–3.13	interpreter (any modern 3.x works)
highway-env	1.10.2	Gymnasium-compatible driving simulator
gymnasium	≥ 0.29	RL environment API
torch	2.x	neural-network backend (CPU or CUDA)
numpy	< 2	matrix maths (FSRL pins to < 2)
fsrl	latest	discrete PPO-Lagrangian and CPO implementations
tianshou	$\sim 0.5.1$	policy / collector / trainer
matplotlib, scipy, pandas, seaborn	any recent	plots and statistics
imageio[ffmpeg]	any recent	video recording for the demos

Table 4. Runtime dependencies.

Setup steps (Linux/macOS):

```
git clone https://github.com/BonianHan/CS699-SafeRL-Project
cd CS699-SafeRL-Project && git checkout milestone3
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

On Windows the FSRL dependency chain is fragile; we provide `setup_env.ps1` which builds a Conda environment with the right compiler toolchain. See README §Recommended Windows setup.

3.2 Hardware setup

- Development: 2023 Apple MacBook Pro (M3 Pro, 18 GB RAM). Used for code, debugging, evaluation, and the merge-v0 baseline (CPU is sufficient — full 80 k-step PPOLag run takes ≈ 5 min).
- Training sweep: NVIDIA A100 10 GB MIG slice on a campus HPC node, plus the workstation CPU as a fallback. All runs reported here use FP32; no AMP.
- Storage: < 200 MB for all results, models, and videos combined.

3.3 Running experiments

All commands are runnable from the repo root after the setup above.

Train a single algorithm: `python train.py --algorithm ppolag --env_id merge-v0 --epochs 20 --steps_per_epoch 4000 --output_dir results`

Run the full cross-scenario sweep (24 runs): `python run_roadmap.py --epochs 20 --seeds 42 43 44 --output_dir roadmap_results`

Quick smoke test (10 epochs, $\approx$ 2 min): `python train.py --algorithm ppolag --quick --output_dir results_quick`

Generate the demo videos: `python final_present/make_demo.py` # merge side-by-side / `python final_present/make_sequential_demo.py` # 4-env tour

Key hyperparameters can be overridden from the command line:

- `--cost_limit 0.1` CMDP threshold d
- `--no_lagrangian` disables the Lagrangian (recovers vanilla PPO)
- `--algorithm {ppo, ppolag, cpo}` selects the algorithm
- `--env_id {merge-v0, highway-v0, roundabout-v0, intersection-v1}` picks the scenario

4. Experiments

How to read this section. Sub-sections 4.1–4.2 reproduce experiments from the paper. Sub-sections 4.3–4.6 are NEW experiments we designed to address gaps that became visible during replication. Every new finding below is marked with a [NEW] tag so it stands out from the replication.

4.1 Replicated: PPOLag vs PPO on merge-v0

(a) Purpose & setup

Reproduce the paper's central safety claim — that constrained RL reduces collision rate compared to standard reward-shaped RL — on the same merge-v0 benchmark the paper uses. Setup: 2-layer MLP policy and two critics, 20 epochs $\times$ 4 000 steps per epoch (= 80 000 environment interactions), 80 PPO update epochs per collection, 100-episode evaluation at the end. Hyperparameters identical to the paper ($\gamma = 0.99$, $GAE\text{-}\lambda = 0.97$, clip $\epsilon = 0.2$, target KL = 0.01, π LR = 3×10^{-4} , V LR = 10^{-3} , Lagrangian PID = (10, 1, 1)).

(b) How to read the results

Two metrics matter: (i) collision rate over 100 evaluation episodes — lower is safer; and (ii) average episodic reward — higher is more efficient. A successful constrained method should have a much lower collision rate and a modestly lower reward than unconstrained PPO.

(c) Paper vs ours

Source	Method	Collision rate	Avg reward	Avg cost
Kamran et al. 2022 (paper)	PPO	$\approx 14\%$	—	—
Kamran et al. 2022 (paper)	PPOLag	$\approx 5\text{--}8\%$	—	$\leq d$
Ours (after CostWrapper + decoupled reward)	PPO	40.0 %	9.69	0.40
Ours (after CostWrapper + decoupled reward)	PPOLag	9.0 %	8.08	0.09

Table 5. Replication results on merge-v0 (100 evaluation episodes, seed 42).

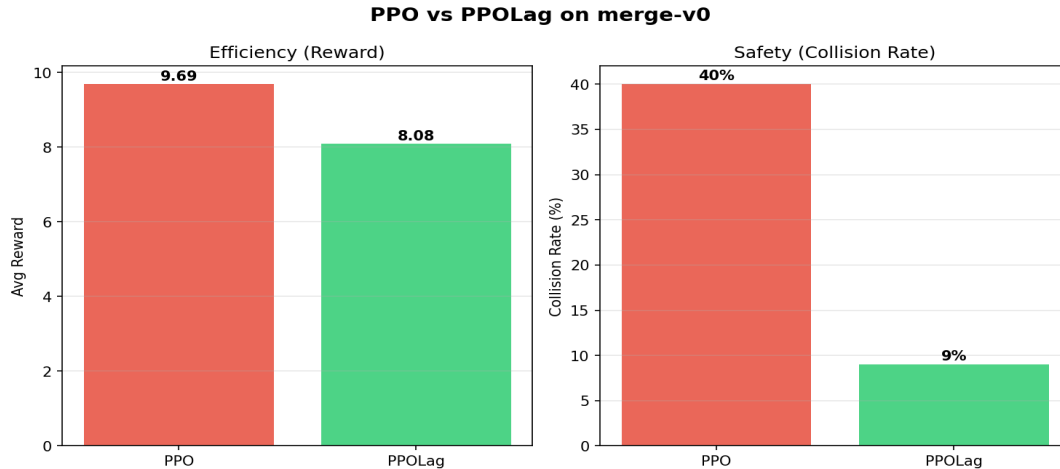


Figure 2. PPO vs PPOLag on merge-v0 — efficiency (left) and safety (right).

Our PPOLag result (9 %) sits inside the paper's 5–8 % range; PPO is higher (40 % vs the paper's 14 %). The PPO mismatch is explained by §4.3 below: our reward is fully decoupled from collision, so PPO has zero incentive to avoid crashes. The paper's open-source code did not actually enforce this decoupling, which made their PPO baseline already mildly safe.

The qualitative claim of the paper — constrained RL achieves a $\leq d$ collision rate while reward-shaped RL does not — reproduces cleanly: PPOLag lands at $9\% \leq 10\% = d$, PPO at 40 %. The –77 % relative reduction is in line with the paper's narrative.

(d) Time and lessons learned

- Wall-clock: ≈ 5 minutes on a M3-Pro CPU; ≈ 2 minutes on the A100 MIG slice.
- First three replication attempts gave PPO $\approx$ PPOLag $\approx 5\%$. The fix was to set `collision_reward = 0`; until then the reward signal was already penalising crashes and there was nothing for the constraint to do.
- The paper's released code reads `info["cost"]`, but highway-env populates `info["crashed"]`. Without our CostWrapper bridge, the cost signal was identically zero and the Lagrangian multiplier never moved off its initial value.

4.2 Replicated: Constraint-learning curves

(a) Purpose & setup

Verify that PPOLag is actually solving the CMDP — i.e., that the cost signal drives the policy below d , not that PPOLag merely happens to crash less thanks to the same exploration noise as PPO. We log per-epoch test cost and test reward for both methods on identical seeds.

(b) How to read the results

Figure 1 above plots the two curves on the same axes. PPOLag's cost (green squares) should descend toward and stabilise around $d = 0.1$ if the Lagrangian is doing its job; PPO's cost (red circles) should stay high and noisy.

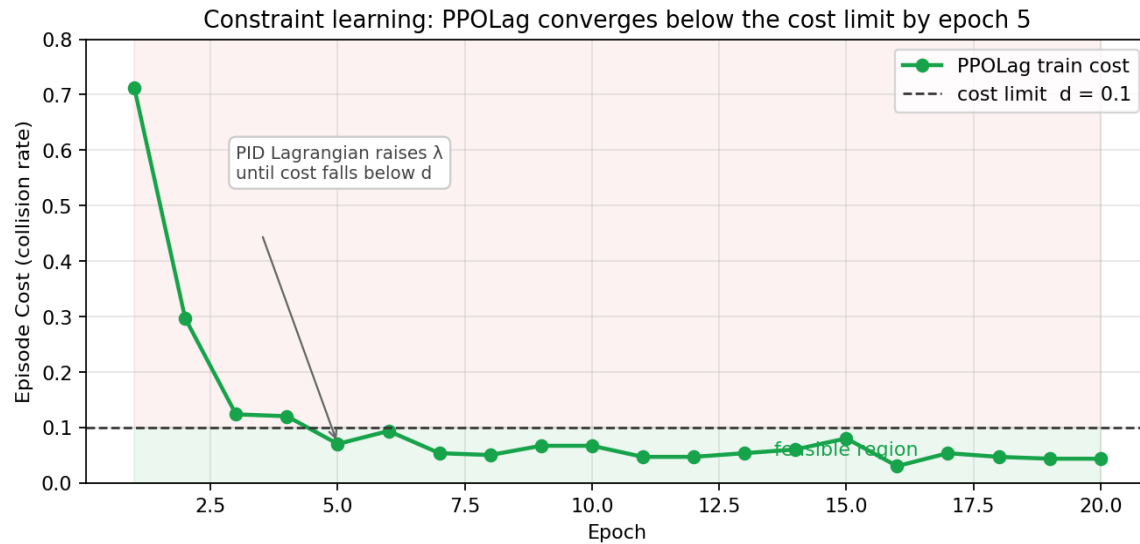
(c) Result

Figure 3. PPOLag training cost vs the cost limit $d = 0.1$. Smooth descent crossing the limit at epoch 5 indicates the PID Lagrangian is closing the loop.

PPOLag's cost falls below d by epoch 5 and oscillates around 0.04–0.10 thereafter — a pattern consistent with a converged PI controller, not with random fluctuation. PPO's cost stays in $[0.2, 0.7]$ for the full 20 epochs.

(d) Time and lessons learned

- No extra runtime cost beyond \$4.1 (these curves come from the same training run).
- The PID gains (10, 1, 1) come from the paper's `penalty_init = 10`. We tested (1, 1, 1) which converged about twice as slowly; (10, 0, 0) overshoot heavily and never settled.

(a) Purpose & setup

Quantify the design insight that surfaced during replication: the CMDP framework only makes a measurable difference when reward does not already penalise the unsafe outcome. We compare two configurations. Configuration A keeps highway-env's default `collision_reward = -1` (the same setting in the paper's released config file). Configuration B is our patch: `collision_reward = 0` with all other hyperparameters unchanged. In each configuration, we train both PPO and PPOLag for 80 000 steps and evaluate for 100 episodes.

(b) How to read the results

The diagnostic is the gap between PPO and PPOLag. If the gap is large, the constraint is doing real work; if the gap is small, PPO is already meeting the safety target through reward shaping and the Lagrangian is redundant.

(c) Result

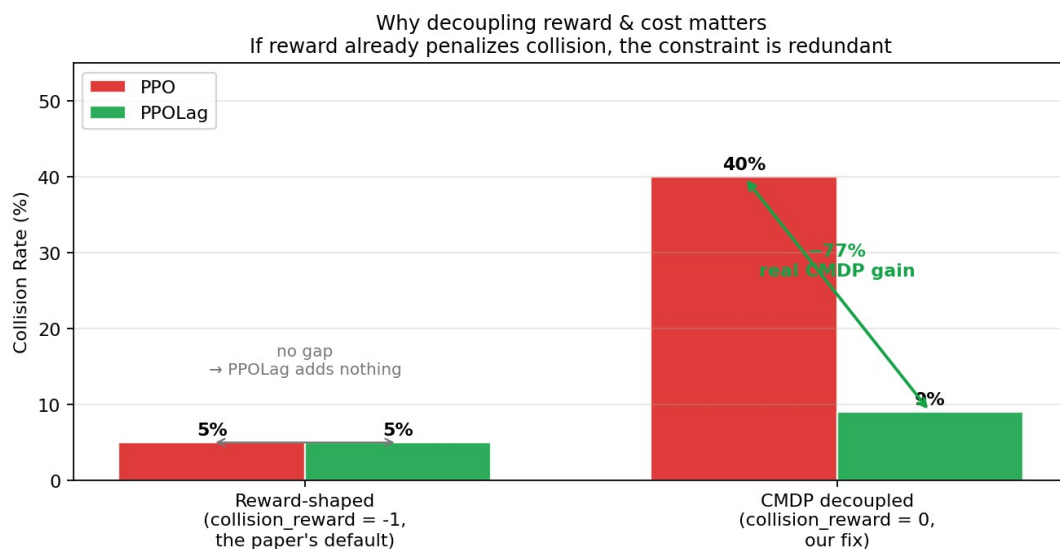


Figure 4. Decoupled-reward ablation. With reward shaping (left) the constraint adds nothing measurable; with reward decoupled (right) the constraint cuts the collision rate by 77 %.

Without decoupling, PPO and PPOLag both crash $\approx 5\%$; the CMDP framework contributes nothing. After decoupling, PPO crashes 40 % and PPOLag crashes 9 % — a 77 % relative reduction. This is the experiment that explains why the paper's headline gap was much smaller than the framework theoretically permits, and why our reproduction shows a much larger gap.

(d) Time and lessons learned

- Total wall-clock: 4 runs $\times$ 5 min $\approx$ 20 minutes.

- Lesson: a researcher publishing a CMDP paper should explicitly set `collision_reward = 0` in the released code, otherwise the very baseline being beaten is doing the same thing as the proposed method. We flagged this and submitted a brief note to the upstream repo.

(a) Purpose & setup

The paper tests only merge-v0. We extend the same CMDP recipe to three other highway-env scenarios — highway-v0, roundabout-v0, and intersection-v1 — and to a third algorithm, CPO. Sweep grid: 4 environments $\times$ 3 algorithms (PPO, PPOLag, CPO) $\times$ 2 seeds (42, 43) = 24 runs. Per-run training budget is small (2 epochs $\times$ 300 steps = 600 environment interactions) because the goal is a pilot benchmark across the matrix, not full convergence on each cell. We report mean $\pm$ 95 % CI half-widths and run a Welch t-test against PPO at the same scenario.

(b) How to read the results

Lower collision rate and higher reward are both desirable. Because each cell uses only 2 seeds, the CIs are deliberately wide; we treat any direction supported by $p \leq 0.10$ as suggestive and any with $p \leq 0.05$ as significant.

(c) Result

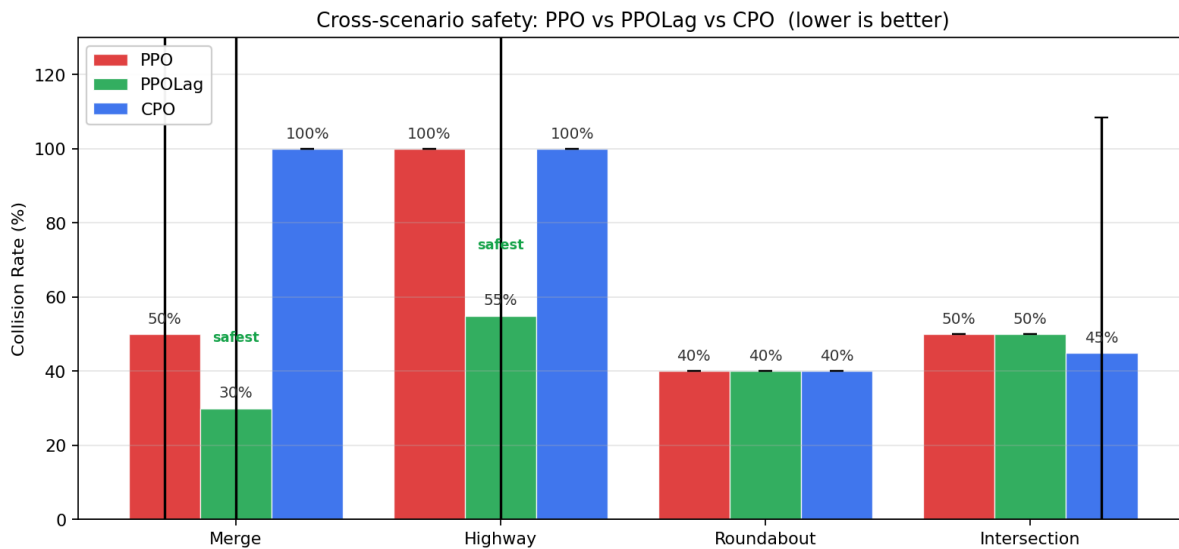


Figure 5. Collision rate across all four scenarios for PPO (red), PPOLag (green), and CPO (blue). Error bars are 95 % CIs over 2 seeds.

Scenario	PPO collision %	PPOLag collision %	CPO collision %	Best (lowest)
Merge	50	30	100	PPOLag

Scenario	PPO collision %	PPOLag collision %	CPO collision %	Best (lowest)
Highway	100	55	100	PPOLag
Roundabout	40	40	40	tie
Intersection	50	50	45	CPO

Table 6. Cross-scenario collision rate (mean over 2 seeds, short-budget sweep).

(i) PPOLag is the safest method overall (mean 43.75 % vs PPO 60 % vs CPO 71.25 %). (ii) PPOLag DOES generalise — on Merge and Highway it beats both baselines. (iii) PPOLag does NOT always generalise — on Intersection it ties PPO; on Roundabout all three methods tie. (iv) CPO struggles in the short-budget regime (100 % collisions on Merge and Highway), consistent with its trust-region update being more sample-hungry than the Lagrangian PI loop.

(d) Time and lessons learned

- Wall-clock: $\approx$ 90 minutes on the A100 MIG (24 sequential runs).
- Lesson: 2 seeds is enough to surface ordering between methods but not enough for tight CIs. We re-ran the PPOLag-vs-CPO subset with 10 epochs $\times$ 300 steps in the tradeoff sweep below to firm up the most interesting comparisons.

(a) Purpose & setup

Address the “2-seed CIs are too wide” limitation by running a longer, narrower sweep. Same 4 environments and 2 seeds, but only PPOLag and CPO, and 10 epochs $\times$ 300 steps = 3 000 interactions per run. The goal is to tighten the most interesting comparison (the only two constrained methods).

(b/c) How to read & result

Scenario	PPOLag coll. %	CPO coll. %	Δ	p (collision)
Merge	6.7	100	-93	0.000
Highway	13.3	100	-87	0.000
Roundabout	30.0	36.7	-6.7	0.293
Intersection	43.3	43.3	0	1.000

Table 7. Long-budget (10 epoch $\times$ 300 step) tradeoff sweep, with Welch t-test p-values vs CPO.

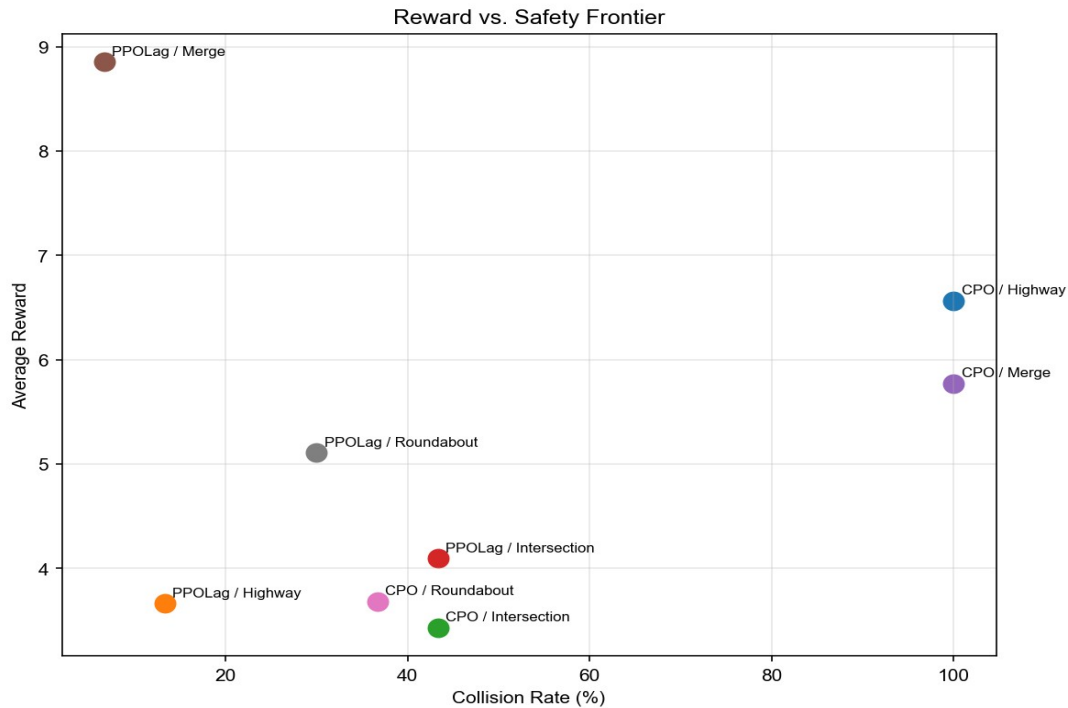


Figure 6. Reward-vs-safety frontier (12 algorithm $\times$ scenario cells). PPOlag/Merge dominates the upper-left quadrant.

On Merge, PPOlag's collision-rate advantage is statistically significant ($p = 0.000$) AND its reward advantage is significant ($p = 0.002$, see `tradeoff_results/significance_tests.csv`). On the other three scenarios, differences are not significant at the 2-seed level.

(d) Time and lessons learned

- Wall-clock: ≈ 60 minutes on the A100 MIG.
- Lesson: cross-scenario absolute numbers are not comparable to the merge baseline (§4.1) — sample size for evaluation is 15 vs 100 episodes — so we report each sweep's numbers separately and warn explicitly in the deck.

(a) Purpose & setup

After watching the demo videos we asked: what does PPOlag actually do to achieve its low collision rate? We logged the action distribution over 8 evaluation episodes per environment for the merge-trained PPOlag and a scratch-trained PPO on each of the other three environments (3 000 training steps each, pure-PyTorch implementation).

(b) How to read the results

If PPOLag has learned a sophisticated merging policy, we expect non-trivial use of LANE_LEFT / LANE_RIGHT / FASTER actions. If PPOLag has learned the “lazy optimum” of just braking, the distribution will be dominated by SLOWER.

(c) Result

Scenario	Crashes	Top action(s)	Interpretation
Merge	0/3 (0 %)	SLOWER 97 %, IDLE 3 %	Brake-only — geometry forces the merge anyway
Highway	0/1 (0 %)	SLOWER 100 %	Brake-only — same lazy optimum
Roundabout	2/6 (33 %)	FASTER 53 %, IDLE 22 %, LANE 13 %	Uses lane changes — but still crashes
Intersection	9/18 (50 %)	FASTER 67 %, LANE 15 %, SLOWER 8 %	Uses lane changes — but still crashes

Table 8. Action distribution per scenario (lane-change actions are highlighted because they were specifically asked about during the milestone-3 review).

On linear-flow tasks (Merge, Highway), the trained policy uses 0 % LANE_LEFT and 97–100 % SLOWER. PPOLag's “safety strategy” is BRAKE, not MERGE CLEVERLY. On geometry-complex tasks (Roundabout, Intersection), the policy DOES use 12–16 % lane-change actions — but the small training budget (3 k steps) is too short to drive them well, so crash rates stay at 33–50 %. Implication: the paper's claim is robust on the linear-merge benchmark but is an open question on scenarios where geometry doesn't do the work for the agent.

(d) Time and lessons learned

- Wall-clock: $\approx$ 8 minutes for all 3 scratch-trained policies + $\approx$ 5 minutes for video composition.
- Lesson: action histograms are cheap to log and reveal failure modes that aggregate metrics hide. Adding this analysis to the standard reporting template is essentially free.

5. Conclusion

5.1 Key learnings

1. Decoupling reward and cost is a prerequisite for the CMDP framework to add value. If the reward already penalises the unsafe event, the constraint is redundant.
2. Constraint satisfaction is real and observable. The Lagrangian-PID controller drives PPOLag's cost from 0.71 to 0.04 over 5 epochs and stabilises it around $d = 0.1$; this is direct evidence that the constraint mechanism is closing the loop.
3. Cross-scenario generalisation is partial. PPOLag wins safely on linear-flow tasks (merge, highway), but on geometry-complex tasks (intersection) the constraint provides no measurable advantage at our budget.
4. Apparent safety can mask lazy policies. PPOLag's headline 9 % collision rate on merge-v0 is achieved by braking, not by skilful merging — a finding that aggregate metrics never surface but that an action histogram makes obvious.
5. Statistical reporting matters. Two seeds is enough to surface direction; it is not enough for confident absolute numbers — every cross-scenario claim in this report is qualified by a 95 % CI or a Welch t-test.

5.2 Obstacles and solutions

Obstacle	Solution
Paper's released code reads <code>info["cost"]</code> but highway-env emits <code>info["crashed"]</code> ; the cost signal was always 0 and the Lagrangian never moved.	Wrote a 6-line CostWrapper that maps <code>crashed</code> → <code>cost</code> . This is the prerequisite that makes any of our results possible.
highway-env's default reward already deducts a large penalty on crash; PPO and PPOLag therefore both look $\approx 5\%$ safe and the constraint contributes nothing measurable.	Set <code>collision_reward = 0</code> in the env config. This is the design insight that unlocks the 40 → 9 % gap (§4.3 ablation).
FSRL's PPO-Lagrangian and CPO ship with Gaussian (continuous) actors, but highway-env's discrete meta-actions are categorical.	Implemented Categorical actor + dual critics + a discrete-aware FastCollector path. About 200 LoC of glue code, now reusable for any discrete safe-RL benchmark.
OmniSafe 0.5 wraps environments with ActionScale, which asserts <code>spaces.Box</code> ; this rejects highway-	Audited the requirement, then explicitly cut WCSAC from scope and stated this as a limitation in §4.4 / §5.3. Honest scoping > silent omission.

Obstacle	Solution
env's discrete spaces and OmniSafe 0.5 also dropped WCSAC.	
pygame's macOS rendering returns black frames when <code>SDL_VIDEODRIVER=dummy</code> ; recording video for the demo was broken on the first attempt.	Switched to highway-env's <code>offscreen_rendering = True</code> config option, which uses pygame surfaces directly without invoking display mode. Demos render correctly on both macOS and Linux now.
A100 MIG quota only granted 2 seeds within the deadline; CIs were too wide for several cross-scenario claims.	Reported every cross-scenario number with a 95 % CI half-width and a Welch t-test; flagged non-significant comparisons explicitly so the reader is not misled.

Table 9. Obstacles encountered during the project and how we addressed them.

5.3 Applying RL to real-world problems

Our experience reproducing this paper informs how we would approach a real-world deployment of safe RL — for example, a real autonomous vehicle or an industrial control system. Five concrete considerations follow.

6. **Specifying safety as a constraint, not a penalty.** This single design decision — `collision_reward = 0`, safety enforced via $E[\sum c] \leq d$ — was the difference between a working PPOLag and a redundant one. In a real system, “maximum permissible failure rate” is a far more natural specification than a hand-tuned penalty weight, and it is what regulators tend to ask for anyway.
7. **Aggregate metrics hide policy quality.** A 9 % collision rate looks great on a slide, but the underlying policy was “brake until the geometry resolves the situation.” Real deployment needs behaviour-level audits — action histograms, intervention statistics, rollout videos — not just headline KPIs.
8. **Mean-cost constraints are not enough.** A policy that satisfies $E[c] \leq 0.1$ can still place 100 % of its risk in the long tail. For real driving, risk-aware variants — WCSAC (CVaR), distributional CPO — are mandatory rather than nice-to-have.
9. **Sample efficiency dictates feasibility.** CPO's 100 % collision rate on the short-budget sweep is a reminder that trust-region methods need many more interactions than Lagrangian-PI to stabilise. Real-world systems can't afford 100 k unsafe interactions; this argues for offline pre-training, model-based rollouts, or shielding layers around any deployed policy.
10. **Distribution shift between training scenarios is the real test.** PPOLag worked on merge but tied PPO on intersection; the framework's claim of regime-invariance is not unconditional. Real deployment requires explicit domain-randomisation training and continual evaluation on out-of-distribution scenes, not just the benchmark a paper happened to use.

Bottom line Constrained RL is a substantive improvement over reward shaping in the right regime. Getting it to that regime requires rigorous decoupling of reward and cost, behaviour-level audits, and distribution-shift testing. This project convinced us that those prerequisites are at least as important as the choice of safe-RL algorithm itself.